AI-assisted testing interview questions, answered with this project

Questions about using large language models (LLMs) in test automation that come up in SDET, QA lead and test automation interviews. Each answer has a short explanation and ends with an **In the project** pointer to where this repository does it (or why it does not). Links go to github.com/niranjankagri/ai-assisted-test-automation.

Versions: Java 21, Anthropic Java SDK 2.73, Claude Opus 5.5 (`claude-opus-5-5`), Playwright for Java 1.63, TestNG 7.12.

Contents

1. LLM basics for testers (Q1-Q14)
2. Where AI helps in testing, and where it does not (Q15-Q24)
3. Generating tests with guardrails (Q25-Q40)
4. Prompting for test automation (Q41-Q50)
5. Review, ownership and risk (Q51-Q60)
6. AI failure triage (Q61-Q68)
7. Reproducibility, cost and CI (Q69-Q74)
8. Scenarios (Q75-Q82)
9. Quick reference

---

1. LLM basics for testers

Q1. What is a large language model, in one paragraph a tester can use?

A model trained on a very large amount of text and code that predicts the next piece of text given what came before. Trained further on instructions and feedback, it follows requests: summarise this, write this test, explain this error. It does not look facts up unless it is given tools or documents; it produces the most plausible continuation, which is usually right for common tasks and confidently wrong for some.

In the project: Claude Opus 5.5 through the Anthropic Messages API, called from Java.

Q2. What are tokens, and why should a tester care?

Models read and write text as tokens (word pieces, roughly 3-4 characters of English each). Prices, limits and speed are all in tokens: the context window, the maximum answer length, the bill. Sending a 2,000-line page object costs far more than sending a 50-line one.

In the project: every live call logs input, cached and output tokens and an estimated cost to `target/ai-usage.csv` and prints them.

Q3. What is the context window?

The maximum number of tokens the model can take in one request: system prompt, conversation, documents and the answer together. Anything not in the context does not exist for the model. Modern

models accept very large contexts, but cost and attention still favour sending only what the task needs.

In the project: the generator sends seven framework files and one test case, not the whole repo (`ProjectFiles.GENERATION_CONTEXT`).

Q4. What is a system prompt, and how is it different from the user message?

The system prompt sets the role, rules and context for the whole conversation (written by the application owner); user messages carry the individual request. Stable instructions belong in the system prompt, the changing input in the user message, which also helps prompt caching.

In the project: `prompts/generate-system.md` + framework code as system blocks; the test case as the user message.

Q5. What is "hallucination", and what does it look like in test automation?

The model states something that is not true, in a confident tone. In test code: calling a page object method that does not exist, asserting a message text the app never shows, inventing a test step the case did not have, or claiming coverage it does not have.

In the project: the compiler catches invented methods, three real runs catch invented texts, and the `coverage` list next to the case's expected results helps the reviewer catch invented or missing checks.

Q6. Are LLM answers deterministic?

Not in general: the same request can produce different answers, and many current models no longer expose a temperature setting to make them more repeatable. Tests that depend on an LLM's live answer are therefore not reproducible. The fix is to record answers and replay them, or to test properties of the answer instead of exact text.

In the project: `AiGateway` records every live answer and replays it in CI (`ai/recordings/`).

Q7. What are structured outputs?

An API feature that constrains the answer to a JSON schema you supply. Instead of parsing free text ("the class name is probably on line 2"), you get JSON with every required field, which an SDK can map straight onto a typed object.

In the project: `outputConfig(DraftTest.class)` and `TriageVerdict.class`: the Java SDK derives the schema from the records.

Q8. What is prompt caching?

The API can cache a prompt prefix (for example a long system prompt plus documents). Later requests with the same prefix read it from the cache at a fraction of the price and faster. Any change in the prefix invalidates the cache, so stable content goes first and changing content last.

In the project: the last system block (framework code) carries a cache breakpoint; the test case comes after it.

Q9. What do "thinking" and "effort" mean on current Claude models?

Models can reason internally before answering ("adaptive thinking"). The effort setting (low to max) controls how much: more effort usually means better answers on hard tasks and more tokens. Claude Opus 5.5 always thinks; effort is the control, with medium as the default.

In the project: test generation runs at high effort, triage at medium.

Q10. How do you choose a model for a testing task?

By the cost of a wrong answer and the volume. Writing a test that a person reviews, or triaging a failure: use a capable model. Classifying thousands of log lines: a fast, cheap one may be enough. Measure on your own examples before deciding.

In the project: Claude Opus 5.5 by default; `AI_MODEL` switches it, and a model change invalidates recordings because the model is part of the key.

Q11. What is a refusal, and how should a tool handle it?

Safety classifiers can decline a request; the API then reports a `refusal` stop reason instead of an answer. A tool must check the stop reason before reading the content and fail clearly instead of treating an empty answer as a result.

In the project: `AiGateway.checkedAnswer` turns `refusal` and `max_tokens` into explicit errors; a server-side fallback model is requested for refusals.

Q12. How do you call an LLM from a Java test framework?

With the provider's official SDK: build a client from the environment (API key), build a request (model, max tokens, system, messages, output format), send it, read the typed response. Keep the call in one class so recording, cost logging and error handling live in one place.

In the project: `AnthropicOkHttpClient.fromEnv()` and `client.messages().create(params)` in `AiGateway`.

Q13. Where does the API key live?

In an environment variable or a secret store, never in the repository, never in logs. CI gets it as a secret if a job really needs it.

In the project: `ANTHROPIC_API_KEY` from the environment; no CI job needs it because CI replays recordings.

Q14. What does a call cost, roughly?

Price per million input tokens and per million output tokens, different per model; cached input is much cheaper. Generating one test with a few thousand tokens of context costs cents, not dollars, but repair rounds and large contexts multiply it.

In the project: the cost estimate per call uses the model's list prices and appears in each draft's `review.md`.

---

2. Where AI helps in testing, and where it does not

Q15. Name useful places for AI in a test team.

Drafting automated tests from written cases, drafting test cases from requirements, generating test data, summarising and classifying failures, explaining unfamiliar code or logs, reviewing test code for smells, and turning bug notes into clear tickets. All of them produce a draft for a person.

In the project: two of these: test drafting and failure triage.

Q16. Where should AI not decide on its own?

Release decisions, whether a failure is acceptable, what the expected behaviour is (that is a product decision), and merging code into the suite. These need accountability, which a model cannot carry.

In the project: nothing is merged without `ai approve <id> <name>`; triage verdicts are labelled as a starting point.

Q17. Why automate from existing test cases instead of letting the model explore the app?

Test cases encode what the business expects; the app only shows what it currently does. A model that writes tests from the current behaviour will happily lock in a bug as the expected result.

In the project: the input is `test-cases/TC-*.md`; the HTML is only context for understanding the screen.

Q18. Does AI replace page objects and framework design?

No, it depends on them more. A clean page object layer gives the model a small, stable vocabulary to compose tests from; without it, generated tests duplicate selectors everywhere and break together.

In the project: the rules forbid selectors in generated tests; only `LoginPage` and `DashboardPage` know them.

Q19. What about "self-healing" locators?

A tool that silently picks another element when a selector breaks hides UI changes that a person should notice and can turn a failing test into a passing test of the wrong thing. Prefer stable test ids and a clear failure.

In the project: not used on purpose; failures go to triage with the evidence.

Q20. How do you measure whether AI assistance is worth it?

Time from test case to merged test, share of drafts that pass the gate in the first round, review time per draft, defects found by the merged tests, and how many merged AI tests later needed fixes. Compare with the same measures for hand-written tests.

In the project: `review.md` records rounds and cost per draft; `ai status` shows which cases are automated.

Q21. Can AI write the test cases themselves?

It can draft them from requirements and point out missing edge cases, which is useful for review. The expected results still need someone who knows the product to confirm them.

In the project: test cases are written by a person; the generator flags unclear points as `assumptions`.

Q22. Is AI good at generating test data?

Yes for realistic, varied data (names, addresses, edge-case strings) and boundary values once you describe the rules. Keep generated data out of production systems and check it against the actual validation rules.

In the project: not needed: the cases use fixed dates so working-day counts are stable.

Q23. What is the biggest risk of AI-written tests?

Tests that pass but check the wrong thing (or nothing). They add false confidence and are harder to spot than tests that fail.

In the project: the review checklist starts with "every expected result is really asserted" and "the assertions would fail if the feature broke".

Q24. How do you introduce AI assistance to a sceptical team?

Start with a narrow, reviewable task, keep humans approving, make the cost and the hit rate visible, and keep a way to turn it off. Show the gate rejecting bad drafts, not just good drafts passing.

In the project: the gate's own tests show every rule rejecting a bad draft (`QualityGateRulesTest`).

---

3. Generating tests with guardrails

Q25. Walk me through your pipeline.

A plain-language test case and the framework code go to Claude, which returns a draft test as structured JSON. A quality gate checks it in three stages: text rules, compilation, three runs against the real app. Failures go back to Claude for up to two repair rounds. A passed draft lands in `ai-drafts/<id>/` with a review file; a person approves it, and only then it moves into `src/test` with an `@AiDrafted` annotation.

In the project: `TestGenerator` , `QualityGate` , `Approver` .

Q26. Why a gate before human review?

Review time is the scarce resource. A draft that does not compile, uses a sleep or fails when run should never reach a person; the machine can reject it in seconds and even tell the model what to fix.

In the project: the gate stops at the first failed stage and feeds the findings into the repair round.

Q27. What do the rules check, and why text rules?

Package and base class, page objects only, `assertThat` present, no sleeps, no JavaScript, no file, network, process, environment or reflection access, no disabled tests, no catch-all blocks, the test case id, the class name, and one coverage line per expected result. Text rules are fast and their messages are precise enough to send back to the model.

In the project: `QualityGate.RULES` , each with the reason shown in the message.

Q28. Are the rules a security boundary?

No. They are a cheap filter; a determined author could get around regular expressions. The control is the human review before merge, and the code is only run locally or in CI with no secrets. State that openly instead of overselling the filter.

In the project: said in the class Javadoc, the README and the design doc.

Q29. Why compile and run inside the same JVM?

So the draft uses exactly the framework the merged test will use: the real base class, page objects and listener. `javax.tools.JavaCompiler` compiles against the test classpath; TestNG runs the class from a child class loader.

In the project: `QualityGate.compile` and `QualityGate.run` .

Q30. Why three runs?

One green run does not prove stability. Three passes in a row against a fresh app is a cheap flakiness check; one failure stops the gate, because a flaky draft is not a review candidate.

In the project: `QualityGate.RUNS = 3` ; each run has its own evidence folder.

Q31. What happens when the gate fails?

The failed checks go back to the model as a new user turn ("your draft did not pass: ...") with its previous answer as the assistant turn. It returns a complete corrected answer. After two repair rounds the draft is kept as failed, with the findings in `review.md`.

In the project: `TestGenerator.MAX_REPAIRS = 2`.

Q32. What does the reviewer get?

The test case's expected results next to the model's own description of how each is checked, the assumptions it made, page object methods it missed, the gate table, the number of rounds and the cost, plus a four-point checklist.

In the project: `ai-drafts/<id>/review.md`.

Q33. Why make the model list its coverage and assumptions?

It turns hidden decisions into reviewable text. A missing coverage line is visible at a glance, and an assumption ("the test case does not say whether the balance shows before or after approval") is a question for the product owner.

In the project: `DraftTest.coverage` and `DraftTest.assumptions`; the gate checks the coverage count.

Q34. What if the page objects do not support a step?

The model must not work around the rule with a raw selector. It reports the missing method in `missingPageMethods` and leaves the check out with a comment, so a person adds the page object method properly.

In the project: stated in the prompt; selectors in a draft fail the rules.

Q35. How does approval work?

`ai approve TC-103 Your Name` refuses drafts that failed the gate, re-runs the rules (the reviewer may have edited the draft), refuses to overwrite an existing test, and adds `@AiDrafted(model, reviewedBy, reviewedOn)`.

In the project: `Approver.approve`.

Q36. Why mark AI-written tests in the code?

Traceability: when such a test fails or is questioned later, everyone can see who reviewed it and which model wrote it. The failure evidence and the triage report also show the flag.

In the project: `@AiDrafted` and `aiDrafted` in `failure.json`.

Q37. Where do the drafts live before approval, and why not in src/test?

In `ai-drafts/`, which is git-ignored. Putting unreviewed tests into the suite, even disabled, makes the suite's meaning unclear.

In the project: `.gitignore` lists `ai-drafts/`.

Q38. How do you keep generated tests consistent with hand-written ones?

Give the model one hand-written test as the style reference, the same base class and page objects, and enforce the conventions with rules. The approve step re-checks them after edits.

In the project: `LoginTest.java` is in the context as the reference.

Q39. How would you scale this to hundreds of test cases?

Batch the requests (the Batch API halves the price for non-urgent work), keep the shared context cached, run the gate in parallel with one browser per worker, and route only gate-passed drafts to reviewers in small groups.

In the project: sequential, because eight cases do not need more; the design notes say what would change.

Q40. What would make you stop trusting the pipeline?

Merged AI tests that later turn out to check the wrong thing, a falling first-round pass rate after a model change, or reviewers approving without reading. Measure these and keep the ability to switch back to hand-written tests.

In the project: rounds per draft are recorded; approval names the reviewer.

---

4. Prompting for test automation

Q41. What makes a good prompt for writing tests?

Clear role and task, the conventions with the reason behind each, the context the model needs (framework code, one example), what to do when something is impossible, and the exact shape of the answer. Explaining *why* a rule exists generalises better than shouting the rule.

In the project: `prompts/generate-system.md` explains how the test is used and why wrong tests are costly.

Q42. Should prompts be in code or in files?

In files under version control, reviewed like code. They change behaviour as much as code does, and here they are part of every recording key, so a prompt change is visible.

In the project: `src/main/resources/prompts/*.md`.

Q43. Why give one example instead of many?

One good example sets the style; many examples cost tokens and the model may copy their details (data, names) into unrelated tests. Add more only if measurements show it helps.

In the project: one reference test.

Q44. How do you stop the model from guessing?

Give it an explicit, honest way out: report unclear points as assumptions, missing methods as a list, and in triage a NEEDS_HUMAN category. Models guess less when not guessing is an allowed answer.

In the project: `assumptions`, `missingPageMethods`, `NEEDS_HUMAN`.

Q45. What is prompt injection, and does it matter here?

Text that the model reads (a web page, a log, a test case) can contain instructions that try to change its behaviour. In triage the model reads the app's HTML and console, so a malicious page could try to steer the verdict. Treat all such content as data, keep the model's actions limited (it only returns JSON here) and keep a person in the loop.

In the project: the model has no tools; its answers are data that a person reads.

Q46. How do you version and test prompts?

Keep them in git, record answers for a fixed set of inputs, and compare results when the prompt changes (gate pass rate, review findings). For larger use, build an evaluation set with expected outcomes.

In the project: recordings per prompt version; the triage demo is a fixed expected outcome (PRODUCT_BUG).

Q47. Why put the test case in the user message and not in the system prompt?

The system prompt plus framework code is the same for every case and can be cached; the case changes per request and goes after the cache breakpoint.

In the project: `TestGenerator.generate`.

Q48. Long prompt or short prompt?

As long as the task needs and no longer. Context that matters (code, the case) earns its place; repeated warnings in capitals do not. Newer models follow plain, reasoned instructions well and can overreact to heavy emphasis.

In the project: the system prompt is a few short paragraphs plus a list of conventions.

Q49. How do you give a model a screenshot?

As an image content block (base64 PNG) next to the text. Vision models read UI states, error banners and layout problems from it.

In the project: triage sends `screenshot.png` as an image block.

Q50. Why should the model return the whole file and not a diff in repair rounds?

A complete file is simpler to validate and compile, and avoids errors from applying a patch to a version the model misremembered. For small files the token cost is negligible.

In the project: "return the whole corrected answer in the same format".

---

5. Review, ownership and risk

Q51. Who owns an AI-written test?

The person who approved it, the same as for any code they merge. The model is a tool; accountability cannot be delegated to it.

In the project: `reviewedBy` in `@AiDrafted`.

Q52. What do you look for when reviewing an AI-written test?

That every expected result is asserted on the right element, that each assertion could fail, that the steps match the case, that test data and users are right, that nothing is checked only by a comment, and that names read well.

In the project: the checklist in `review.md`.

Q53. What data may you send to an external model?

Only what policy allows. Source code, test cases and screenshots can contain confidential or personal data; check the provider's data handling terms and your company's rules, and strip secrets and personal data

before sending.

In the project: demo app with demo users only; the README and code send no credentials.

Q54. Can generated code be dangerous to run?

Yes: it could delete files, call networks or read secrets, by mistake or through injected instructions. Filter for obvious patterns, run without secrets, and require review before it enters the suite.

In the project: rule "no file, network, process or reflection access"; CI holds no API key.

Q55. How do you prevent over-trust?

Make uncertainty visible (assumptions, confidence, NEEDS_HUMAN), show the evidence behind each verdict, label AI output as a draft, and audit a sample of approved work.

In the project: triage asks for quoted evidence and a confidence level.

Q56. What about licensing of generated code?

Follow your company's policy and the provider's terms; for test code built on your own framework the risk is low, but the policy decides.

In the project: MIT-licensed repo; generated tests use only this repo's framework.

Q57. How do you keep the suite understandable when part of it is AI-written?

Same conventions, same review bar, visible marking, and no generated test without a linked test case.

In the project: `@TestCase` is required by the rules.

Q58. What does the gate not catch?

Wrong expectations that the app happens to satisfy, assertions on the right text in the wrong place, missing negative checks, and tests that pass for the wrong reason. That is exactly what the reviewer is for.

In the project: listed in the design doc's gate table as "high cost of a wrong decision: review".

Q59. How do you handle a disagreement between the test case and the app?

That is a finding, not a test-writing problem: either the case is wrong or the app is. The model should not "fix" it by asserting the current behaviour.

In the project: the prompt asks for exact expected texts from the case; triage separates product and test bugs.

Q60. What would you never automate with AI?

Decisions with accountability (release go/no-go), changes to production data, and anything that cannot be reviewed or reverted.

In the project: the tools only write drafts and reports.

---

6. AI failure triage

Q61. What does your triage do?

For each failed UI test it sends the evidence (error and stack, console, URL, DOM, screenshot) plus the test case and the test code, and gets back one of five categories with a confidence, quoted evidence, a next step

and, for product bugs, a draft ticket.

In the project: `FailureTriage`, output in `target/ai-triage/triage.md`.

Q62. Which categories, and why five?

PRODUCT_BUG, TEST_BUG, ENVIRONMENT, FLAKY_OR_TIMING and NEEDS_HUMAN. They map to who acts next: developers, QA, infra, QA with a stability fix, or a person first.

In the project: `TriageVerdict.Category`.

Q63. What evidence do you collect on failure, and why those?

What a person would open: the error and stack (what failed), the DOM and a screenshot (what the page showed), the console (app errors), the URL (where), and the Playwright trace (how it got there).

In the project: `FailureEvidenceListener` + trace from `BaseUiTest`.

Q64. How do you know the triage works?

Plant a known product bug and check the verdict, and check a known test bug the same way. Keep these as regression cases when the prompt or model changes.

In the project: `-Dleavedesk.bug=true` switches the overlap rule off; the CI triage demo fails unless the verdict is PRODUCT_BUG.

Q65. Why ask for a confidence and quoted evidence?

A verdict without evidence cannot be checked; a confident wrong verdict sends people the wrong way. Quotes let a person verify the reasoning in seconds.

In the project: `TriageVerdict.evidence` and `confidence`.

Q66. Should triage file bugs automatically?

Not without a person: the draft ticket saves writing time, but someone confirms the bug and its severity first.

In the project: the ticket is Markdown in the report; nothing is sent anywhere.

Q67. How do you stop triage from becoming expensive on a red build?

Group failures with the same error first, cap the HTML size, skip triage when the whole run failed for an environment reason that a cheap check detects, and run triage only on CI failures, not locally on every run.

In the project: HTML capped at 20,000 characters; one call per failure.

Q68. How does triage stay reproducible?

Record the answers and key them on normalised evidence (ports, retry counts and timings removed), so the same failure maps to the same recording.

In the project: `FailureTriage.stable`.

---

7. Reproducibility, cost and CI

Q69. How do you run AI features in CI without a key and without flakiness?

Record answers locally with a key, commit them, and replay them in CI. The recording key covers everything that should invalidate an answer; a change makes replay fail with a clear message to re-record.

In the project: `AI_MODE=replay`; CI jobs `ai-replay` and `triage-demo`.

Q70. What goes into a recording's key?

The model, the system prompt, the test case and the framework code sent as context; for repair rounds the draft being repaired; for triage the normalised error, HTML, test code and case.

In the project: `AiGateway.recordingFile`: SHA-256 over model and key material.

Q71. What does the replay job prove?

That the recorded drafts still pass today's gate against today's app: if someone changes the app or the framework so that an approved AI test would no longer be generated correctly, CI shows it.

In the project: `ai generate --approved` in the `ai-replay` job.

Q72. How do you keep the cost under control?

Small, relevant context; prompt caching for the shared part; the right effort level per task; stopping repair rounds after two; recording so a question is paid for once; and a visible cost per call.

In the project: all six.

Q73. What do you do when the provider is down or rate-limited?

The SDK retries transient errors with backoff. Beyond that, the tooling should fail clearly and the normal test suite must not depend on the provider at all.

In the project: the UI suite never calls the API; only `generate` and `triage` do.

Q74. How do you test the AI tooling itself?

Unit-test everything around the model: the rules (each one rejecting a crafted bad draft), compile and run stages with hand-made drafts, replay, the approve marking, evidence normalisation. The model's answers are tested through recordings and fixed expected outcomes.

In the project: `QualityGateRulesTest`, `QualityGateRunTest`, `ToolingUnitTest`.

---

8. Scenarios

Q75. A generated test passes, but you suspect it checks nothing. How do you find out?

Break the feature and run it (here: the planted bug for overlap), or change the expected text in the test and confirm it fails. A test that stays green against a broken feature is not a test.

In the project: the gate's own test changes an expected text and checks that the run fails.

Q76. The first-round gate pass rate drops after a model upgrade. What do you do?

Compare old and new recordings for the same cases, read the gate findings, adjust the prompt or the rules, and keep the old model until the rate is back. A model upgrade is a change like any other.

In the project: the model is part of the recording key, so old and new answers sit side by side.

Q77. A reviewer approves everything without reading. How would you notice?

Very short review times, approvals of drafts with assumptions nobody answered, and AI tests that later need fixes. Spot-check approved drafts and discuss findings in the team.

In the project: approval records the reviewer and date.

Q78. The test case is ambiguous. What should the pipeline do?

Draft the test with the most reasonable reading, list the ambiguity as an assumption, and let the reviewer ask the product owner. Fix the test case afterwards so the next draft does not need the assumption.

In the project: `assumptions` in `review.md`.

Q79. Triage says TEST_BUG, but you think it is a product bug. Now what?

Read the quoted evidence and check it against the test case. If the model misread it, record the case as a triage regression example and improve the prompt; the verdict was a suggestion, the decision is yours.

In the project: the report says AI verdicts are a starting point.

Q80. Your manager asks for "AI to write all our tests". How do you answer?

Agree on a pilot with a narrow scope (one feature, written test cases, existing page objects), human approval, and measures (time to merged test, first-round pass rate, review time, later fixes), then decide from the numbers.

In the project: this repo is that pilot in miniature.

Q81. How do you explain this project in two minutes?

"Claude drafts Playwright tests from plain-language test cases. A gate rejects drafts that break framework rules, don't compile or are flaky, and feeds the reasons back for two repair rounds. A person approves every merged test, and the approval is recorded in the code. When a run fails, Claude triages each failure with the evidence into product bug, test bug, environment or flaky, with a draft ticket. Answers are recorded, so CI replays them without a key."

Q82. What would you add next?

An evaluation set of test cases with known-good tests to measure drafts automatically, batch generation for large backlogs, grouping of failures before triage, and a link to the test management tool so cases come in and results go back.

In the project: listed as limitations and next steps in the README.

---

9. Quick reference

Topic	Where in the repo
Only Claude call site	`src/main/java/.../ai/AiGateway.java`
Prompts	`src/main/resources/prompts/`
Draft answer schema	`ai/generate/DraftTest.java`
Gate rules and stages	`ai/gate/QualityGate.java`
Review file	`ai-drafts/<id>/review.md`
Approve step	`ai/generate/Approver.java`
Failure evidence	`framework/FailureEvidenceListener.java`
Triage	`ai/triage/FailureTriage.java`, `target/ai-triage/triage.md`
Recordings	`ai/recordings/<purpose>/`

Command	Does
`ai status`	automated / drafted / open per test case
`ai generate TC-103`	draft + gate (+ repair rounds)
`ai gate TC-103`	gate an edited draft again
`ai approve TC-103 Your Name`	move into `src/test` with `@AiDrafted`
`ai triage`	verdicts for the last UI run's failures
`mvnw test -Dsuite=ui -Dleavedesk.bug=true`	run with the planted product bug